


fiche 1. Différences et Points Communs entre Threads et Sous-Processus (Fork)

Différence

- La différence entre un thread et un sous-processus classique réside dans la gestion du segment de données globales. En effet, pour un sous-processus, ce segment est une **copie** de celui de son processus parent, tandis qu'un thread **partage** ce segment avec son géniteur. Cette duplication des ressources pour les sous-processus est coûteuse pour le système d'exploitation, bien plus que la création d'un thread, ce qui justifie l'appellation de **"processus léger"** pour les threads.
- Ainsi, contrairement aux sous-processus, qui sont totalement indépendants les uns des autres, un thread fils dépend de son thread parent. Cela signifie que la **terminaison du thread principal** (initial) d'un processus entraîne également celle de tous ses threads fils.

Points Clés



fiche 2. Notion de Code Ré-entrant

Définition

- Un code ré-entrant peut être exécuté simultanément par plusieurs threads sans provoquer d'interférences.

Explication

- Dans un environnement multithread, plusieurs threads peuvent exécuter simultanément le même code. Pour que cela se passe correctement, le code doit être **réentrant**, c'est-à-dire conçu pour être exécuté simultanément sans risque de conflit.
- Le thread a des états similaires à un sous-processus classique (en attente, prêt, en cours d'exécution, terminé).
- Les problèmes de **synchronisation** des threads et d'**accès concurrent aux ressources** doivent être gérés correctement pour éviter les erreurs. Les mécanismes de synchronisation propres aux threads offrent des solutions spécifiques à ces problèmes.

- **Threads** : Création rapide, moins coûteuse en ressources, mais nécessite une synchronisation (mutex, sémaphores, etc.).
- **Sous-processus (fork)** :
Création plus lente, car nécessite la duplication de la mémoire.
Communication plus complexe via IPC (pipes, sockets, etc.).

Points Communs

- **Exécution concurrente** :
Les threads et les sous-processus permettent tous deux d'exécuter plusieurs tâches en parallèle.
- **Création depuis un processus parent** : Un thread et un sous-processus sont tous deux créés à partir d'un processus existant

Solutions

- Utiliser des variables locales.
- Protéger l'accès aux données partagées avec des mutex ou sémaphores.



fiche 3. Création et Gestion des Threads

Explication

- **Les 4 fonctions essentielles :**

1. `pthread_create()` : Création d'un thread.

? Comment : Indiquer la fonction que le thread doit exécuter et éventuellement lui fournir un paramètre

1. `pthread_exit()` :
Terminaison d'un thread.

? Comment : En terminant la fonction qu'il exécute par un appel de **pthread_exit** qui permet de renvoyer une valeur; *ceci ne fait pas disparaître le thread !*

1. `pthread_join()` : Attente de fin d'un thread.

? Comment : En récupérant sa valeur de retour **[pthread_join]**;

1. `pthread_detach()` :
Détachement d'un thread.

? Comment : Marquer un thread pour la destruction et libérer automatiquement ses ressources lorsque celui-ci termine sa tâche.

détruire définitivement un thread et ainsi récupérer ses ressources



Fiche 4 : Identité d'un Thread

Définition :

Un **thread** s'identifie par trois éléments :

1. **Le pid de son processus père :** C'est l'identifiant du processus dans lequel le thread s'exécute.
2. **Son tid (Thread IDentifier) :** C'est l'identifiant unique du thread à l'intérieur du processus.
3. **OU un label combiné** sous la forme **pid.tid** : Cela permet de donner un identifiant unique à un thread dans le système en combinant ces deux valeurs.

? Comment : Avec les fonctions



```
pid_t getpid (void); //
LE PROCESSUS

pthread_t
pthread_self(void); //
THREAD si y a pas
d'identification, se
sera un code
aléatoire
```

[pthread_detach];

→ le thread

en cours d'exécution n'est pas arrêté

→ il ne

sera détruit que lorsqu'il

aura terminé sa tâche;

cette fonction "marque

pour la destruction" plutôt

qu'elle ne détruit

réellement –

→ elle "détache" le thread

→ elle le rend indépendant

et laisse sa terminaison

libérer ses ressources

automatiquement.



Code POSIX

```
pthread_create(&thread, NULL, fonction,  
NULL);  
pthread_exit(NULL);  
pthread_join(thread, NULL);  
pthread_detach(thread);
```



fiche 5. Mise en Sommeil d'un Thread

Definition

- `nanosleep()` suspend un thread pendant un temps défini.

Explication:

On peut mettre un thread en sommeil comme tout processus, mais par un "sleep" propre aux threads (car le sleep classique n'est pas supporté par les librairies multi-threads)



```
int nanosleep
(<intervalle de temps
- const struct
timespec *>, <temps
restant - struct
timespec *>);
```

Valeur de retour	erreur
0	Succès
-1	Le délai d'attente a été interrompu, errno prend la valeur EINTR , 2e paramètre reçoit la valeur du temps non écoulé .



fiche 6. Fonction de Terminaison d'un Thread



`pthread_cleanup_pop`

◆ **Utilité :**

- Retire une fonction de nettoyage enregistrée avec `pthread_cleanup_push`.
- Peut **exécuter immédiatement** la fonction de nettoyage ou simplement l'enlever sans l'exécuter.

◆ **Exemple concret :**



Imagine que tu cuisines et que tu as mis une alarme pour te rappeler de nettoyer.

- Si tu finis de cuisiner et tu veux nettoyer immédiatement, tu laisses sonner l'alarme et tu nettoies →
`( pthread_cleanup_pop(1) ).`
- Si tu décides finalement que le nettoyage n'est pas nécessaire, tu désactives juste l'alarme sans rien faire →
`( pthread_cleanup_pop(0) ).`

`pthread_cleanup_push`

- **◆ Utilité :**
- Évite les fuites de mémoire et les fichiers laissés ouverts en cas d'interruption brutale.
- Permet d'enregistrer une fonction de nettoyage qui s'exécutera **automatiquement** si le thread est annulé.
- il n'y a pas de valeurs de retour c'est une macro



7. Comportement des Threads vis-à-vis des Signaux

Dans un programme avec plusieurs threads, la gestion des signaux suit des règles précises pour éviter les problèmes et assurer un bon fonctionnement.

1. Qui reçoit les signaux ?

- Un signal peut être **envoyé à un thread en particulier** ou à tout le programme.
- Les signaux **synchrones** (ex: erreur de mémoire `SIGSEGV`) vont toujours au thread qui a causé le problème.
- Les signaux **asynchrones** (ex: `SIGINT` pour arrêter un programme) sont envoyés à un seul thread choisi par le système, sauf si tous les threads l'ont bloqué.

2. Bloquer les signaux

- Chaque thread peut **choisir quels signaux il accepte ou bloque** grâce à `pthread_sigmask()`.
- Un signal bloqué **reste en attente** et sera traité plus tard quand il sera débloqué.

3. Gérer les signaux proprement

- Pour éviter les conflits, il est recommandé de **créer**



fiche 8. Arrêt Programmé d'un Thread

Definition

- `pthread_cancel()` demande l'arrêt du thread



int **pthread_cancel**
(<handle du thread - pthread_t>); //permet à thread de "tuer" un autre thread

Explication:

La manière de prendre en compte une telle demande dépend de la manière dont le thread a été configuré vis-à-vis de ce type d'événement.

Cette configuration est déterminée par deux indicateurs positionnés par

deux fonctions :

- `pthread_setcancelstate()` permet de bloquer ou autoriser l'annulation d'un thread

un thread spécial qui s'occupe des signaux.

- Ce thread peut **attendre les signaux bloqués** avec `sigwait()`, ce qui permet une gestion plus organisée.

4. Exemples d'utilisation

- **Serveurs multi-threadés :**
Un seul thread capte les interruptions pour éviter des arrêts imprévus.
- **Programmes critiques :**
Certains threads peuvent bloquer les signaux pour ne pas être interrompus pendant des tâches importantes.



int
pthread_setcancelstate
(<état donné à
l'indicateur – int>,
<ancien état de
l'indicateur> - int *>);

*//permet à un thread
d'accepter ou non
d'être "cancélé"*

- `pthread_setcanceltype()` définit l'état d'arrêt (Asynchrone, différé).



int
pthread_setcanceltype
(<état donné à
l'indicateur – int>,
<ancien état de
l'indicateur> - int *>);

*//si un thread a accepté
d'être "cancélé", il peut
décider quand*



fiche 9. Techniques de Synchronisation des Threads

- Les différentes techniques de synchronisation des threads (join, mutex, mutex+variable de condition, pause)

1. **pthread_join()** (Attente de fin de thread)

- **Concept** : Permet à un thread de attendre qu'un autre thread termine son exécution.

- **Prototype** : int
pthread_join(pthread_t thread, void **retval);

2. Mutex (Mutual Exclusion)

- **Concept** : Mécanisme de synchronisation qui permet de garantir qu'une ressource partagée est accédée par un seul thread à la fois.

- **Prototypes importants** :

- Initialisation :

```
int
pthread_mutex_init(pthread_mutex_t *mutex, const
pthread_mutexattr_t *attr);
```

- Verrouillage :

```
int
pthread_mutex_lock(pthread_mutex_t *mutex);
```

- Déverrouillage :

```
int
pthread_mutex_unlock(pthread_mutex_t *mutex);
```



Fiche 10: Politique de scheduling

Explication

définir cette priorité, il faut d'abord définir une politique de scheduling. Cette politique se définit en construisant un attribut de thread par :

- `pthread_attr_setschedpolicy()` : Définit la politique.



```
int
pthread_attr_setschedpolicy(pthread_attr_t *attr, int policy);
```

→ Le deuxième paramètre est un entier qui peut prendre les valeurs symboliques définies dans pthread.h :

- **Politique de planification** :

1. **SCHED_FIFO** : C'est comme si "le thread avec la plus haute priorité doit travailler sans s'arrêter, jusqu'à ce qu'il n'ait plus rien à faire". Une fois que le thread finit, un autre thread à haute priorité peut commencer.
2. **SCHED_RR** : Ici, le thread avec la plus haute priorité commence à travailler, mais il n'a pas le droit de rester indéfiniment. Chaque thread à la même priorité aura son tour pour travailler (ils se partagent le temps).

- Destruction :

```
int
pthread_mutex_destroy(pthread_mutex_t
*mutex);
```

3. Mutex + Variable de condition

- **Concept** : Utilisation combinée d'un mutex et d'une variable de condition pour gérer l'attente et la notification entre threads.
- **Prototypes importants** :
 - Initialisation de la condition :


```
int
pthread_cond_init(pthread_cond_t
*cond, const
pthread_condattr_t *attr);
```
 - Attente d'une condition :


```
int
pthread_cond_wait(pthread_cond_t
*cond, pthread_mutex_t
*mutex);
```
 - Notification d'un thread :


```
int
pthread_cond_signal(pthread_cond_t
*cond);
```
 - Destruction de la condition :


```
int
pthread_cond_destroy(pthread_cond_t
*cond);
```

4. nanosleep()

- **Concept** : Permet de suspendre l'exécution d'un thread pendant une durée spécifiée, en secondes et nanosecondes.
- **Prototype** : `int nanosleep(const struct timespec *req, struct timespec *rem);`

3. **SCHED_OTHER** : C'est la règle par défaut, où tous les threads (peu importe leur priorité) ont droit à un peu de temps pour travailler.

→ définir un attribut pour le thread qui spécifiera cette priorité

- `pthread_attr_setschedparam()` : Ajuste priorité.



```
int
pthread_attr_setschedparam(pthread_attr_t
*attr, const struct sched_param
*param);
```





fiche 11. Types de Mutex

Un **mutex** (exclusion mutuelle) est un mécanisme de synchronisation utilisé pour contrôler l'accès à une ressource partagée dans un environnement multithread, en s'assurant qu'une seule tâche (thread) puisse accéder à une section critique à la fois.

1. `pthread_mutex_lock` : Cette fonction est utilisée pour verrouiller le mutex. Si le mutex est déjà verrouillé par un autre thread, l'appelant sera bloqué jusqu'à ce que le mutex soit libéré.
 - Exemple :
`pthread_mutex_lock(&mutex);`
2. `pthread_mutex_unlock` : Cette fonction libère le mutex. Une fois le mutex déverrouillé, un autre thread pourra le verrouiller et accéder à la ressource protégée.
 - Exemple :
`pthread_mutex_unlock(&mutex);`
3. `pthread_mutex_trylock` : Contrairement à `pthread_mutex_lock`, cette fonction essaie de verrouiller le mutex sans bloquer le thread si le mutex est déjà verrouillé. Si

fiche 12: Les trois types de mutex sont :

1. **Mutex normal** : Utilisé par défaut, il empêche un thread de verrouiller un mutex qu'il a déjà verrouillé, provoquant un *deadlock* si tenté à nouveau.
2. **Mutex récursif** : Permet à un thread de verrouiller plusieurs fois un mutex sans provoquer de *deadlock*. Chaque appel à `lock()` doit être suivi de la même quantité de `unlock()` pour libérer le mutex.
3. **Mutex non récursif** : Similaire au mutex normal, mais lorsqu'un thread tente de verrouiller un mutex déjà verrouillé, une erreur est générée plutôt qu'un *deadlock*. Ce type est moins performant et est généralement remplacé par des mutex rapides une fois le développement terminé.

le mutex est disponible, il sera verrouillé, sinon la fonction retourne une erreur.

- Exemple :

```
if (pthread_mutex_t  
rylock(&mutex) !=  
0) {  
    // Mutex déjà ver  
rouillé, on ne bloqu  
e pas.  
}
```

4. **pthread_mutex_destroy** : Cette fonction est utilisée pour détruire un mutex lorsque celui-ci n'est plus nécessaire. Cela libère toute la mémoire allouée pour le mutex.

- Exemple :

```
pthread_mutex_destroy(&mutex);
```

- fiche 13 Utilisation et but d'une variable de condition (paradigmes !!!)

Variables de condition

(pthread_cond_t) : Permettent à un thread d'attendre qu'un événement spécifique se produise, en association avec un mutex (pthread_mutex_t).

Mutex (pthread_mutex_t) : Assure l'atomicité des modifications sur des variables partagées et protège l'accès aux variables de condition.

Création et destruction des variables de condition :

- pthread_cond_init() : Initialisation.
- pthread_cond_destroy() : Libération des ressources.

Attente d'un événement

(pthread_cond_wait) : Bloque le thread tout en libérant le mutex, jusqu'à ce que pthread_cond_signal() soit appelé.

Réveil d'un thread

(pthread_cond_signal) : Réveille un seul thread en attente. Si aucun thread n'attend, le signal est perdu.

Modèle d'utilisation classique :

- Vérification de condition avec pthread_mutex_lock() → Attente via pthread_cond_wait() → Libération du mutex avec pthread_mutex_unlock() une fois l'événement détecté.

Gestion des threads :

- `pthread_create` : Création de threads.
- `pthread_detach` : Détachement pour éviter des fuites mémoire.
- `pthread_exit` : Terminaison d'un thread.

Temporisation (`nanosleep`) : Simule un temps de traitement variable.

fiche 14: Les variables spécifiques aux threads

Principe

Les variables spécifiques aux threads sont des variables dont la valeur est propre à chaque thread d'un programme. Elles permettent d'éviter les conflits entre threads en stockant des données distinctes pour chacun.

Fonctionnement

Chaque thread possède sa propre copie de la variable spécifique, isolée des autres threads. Cette isolation est assurée par des mécanismes comme le stockage local des threads (TLS - Thread Local Storage). L'accès à ces variables est généralement géré via des bibliothèques ou des fonctionnalités intégrées du langage utilisé.

But

L'objectif principal est d'assurer l'indépendance des données entre threads tout en facilitant la gestion de l'état propre à chaque thread. Cela évite les problèmes de concurrence et réduit la nécessité d'utiliser des verrous (mutex).

Utilisation

Les variables spécifiques aux threads sont utilisées dans :

fiche 15 : les routines d'initialisation unique

Principe et fonctionnement

Certaines ressources, comme les clés spécifiques aux threads, mutex ou variables de condition, doivent être initialisées une seule fois. La fonction `pthread_once()` permet de garantir cette unicité :

```
int pthread_once(pthread_once_t *controleur, void (*fonction_init)(void));
```

Le premier thread appelant `pthread_once()` exécute la routine d'initialisation, les autres attendent sa fin. L'état d'exécution est enregistré dans une structure `pthread_once_t`, définie dans `pthread.h` :

```
c
CopierModifier
typedef volatile struct __pthread_once_t {
    long _Pfield(state);
    long _Pfield(reserved)[10];
} pthread_once_t;
```

L'initialisation se fait avec la macro `PTHREAD_ONCE_INIT`, garantissant l'exécution unique :

```
c
CopierModifier
#define PTHREAD_ONCE_INIT
```

- La gestion des contextes locaux aux threads (ex. stockage de caches, gestion de sessions).
- La programmation concurrente pour éviter les conflits d'accès aux ressources partagées.
- L'optimisation des performances en réduisant la synchronisation entre threads.

{PTHREAD_ONCE_UNINIT}

But et utilisation

`pthread_once` évite les conflits liés à des initialisations multiples en contexte multi-thread. Il est utilisé pour garantir qu'une ressource partagée est créée une seule fois.

Exemple de code :

```
c
CopierModifier
void initCle() {
    puts("=== Initialisation d'une
    clé ===");
    pthread_key_create(&cle, NUL
    L);
}

void *lire(void *h) {
    pthread_once(&controleur, init
    Cle); // Exécute initCle une seule
    fois
}
```

Ainsi, même si plusieurs threads exécutent `lire()`, la clé est créée une seule fois.

Gestion des erreurs

`pthread_once` retourne :

- **0** en cas de succès
- **EINVAL (22)** si le paramètre est invalide

fiche 16: Les Modèles Classiques de Threads

Les modèles classiques d'exécution concurrente reposent sur des principes d'organisation des threads qui définissent leur interaction et leur mode de création. Ces modèles permettent d'optimiser les performances des systèmes multi-threadés selon les besoins des applications.

1. Critères de Classification

Les modèles de threads se distinguent principalement selon deux critères fondamentaux :

- **Répartition du code** : Certains modèles attribuent le même code à tous les threads, tandis que d'autres répartissent des rôles spécifiques.
- **Mode de création des threads** :
 - **Statique** : Tous les threads sont créés au lancement du programme, garantissant une performance stable.
 - **Dynamique** : Les threads sont créés et détruits à la demande, permettant une utilisation plus flexible des ressources.

2. Comparaison des Modèles Classiques

Modèle	Code unique ou spécifique ?	Création des threads ?
Modèle Parallèle	Unique	Statique
Modèle Producteur-Consommateur à la demande	Unique	Dynamique
Modèle Producteur-Consommateur en Pool	Unique	Statique
Modèle Pipeline	Spécifique	Dynamique

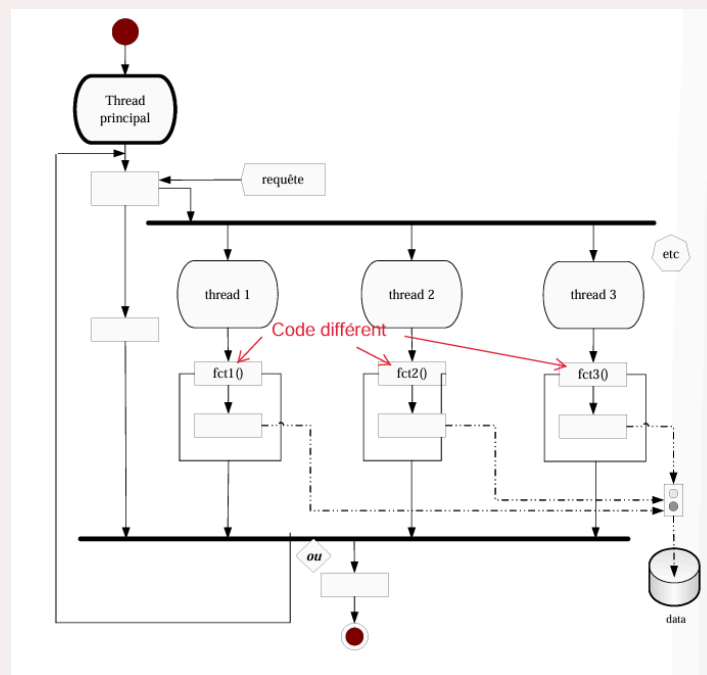


Fiches 17: des Modèles de Threads

1. Modèle Parallèle

- **Explication générale** : Une tâche complexe est divisée en sous-tâches indépendantes, qui sont attribuées à différents threads. Le programme continue une fois que tous les threads ont terminé leurs tâches respectives.

Schéma



- **Éléments de synchronisation nécessaires** :
 - **pthread_join()** pour attendre la fin de chaque thread avant de poursuivre.
 - **Mutex ou sémaphores** peuvent être utilisés pour gérer l'accès aux ressources partagées, bien que non explicitement présents dans cet exemple.
- **Code des fonctions de threads** :
 - **Parallélisme** : Chaque thread exécute une sous-tâche indépendante (par exemple, recherche dans un fichier).
 - **Boucle infinie** : Il n'y a pas de boucle infinie dans ce modèle; chaque thread effectue sa tâche et se termine.
 - **Librairie POSIX** : Utilisation de `pthread_create()` pour créer des threads, et `pthread_join()` pour synchroniser leur fin.

- **Paradigme:** repose sur l'exécution simultanée de sous-tâches indépendantes par plusieurs threads, permettant de diviser et traiter une tâche complexe plus rapidement.

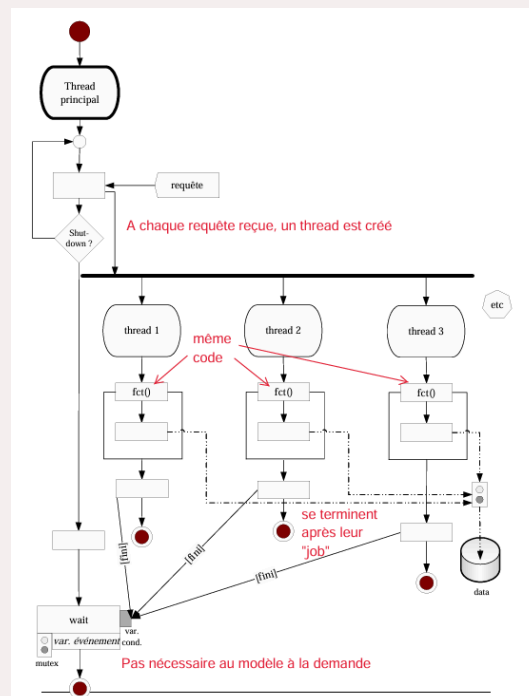
2. Modèle Producteur-Consommateur à la demande

Modèle le plus courant, il s'utilise dans une architecture client-serveur

Un thread particulier (souvent le thread principal ou un thread créé à cet effet) reçoit

des requêtes de divers clients par un moyen quelconque (par exemple, terminal, réseau, ligne de commande, fichier batch, ...).

schéma



- **Création dynamique de threads (= à la demande) :**

Un

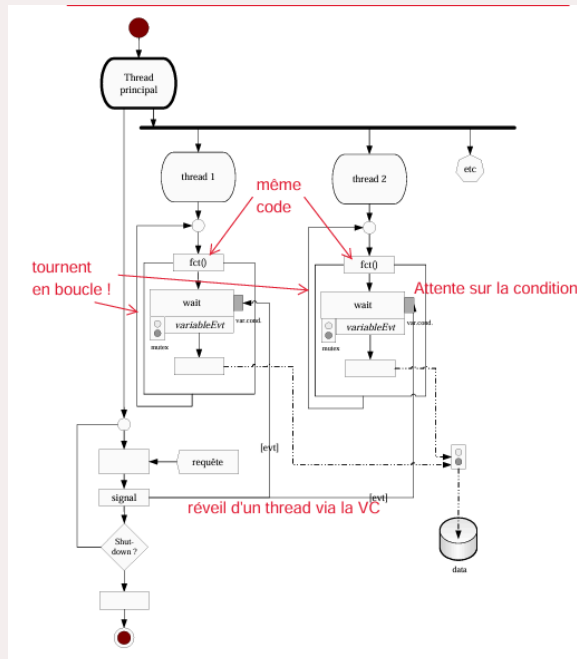
nouveau thread est généré à chaque requête et se termine après traitement.

Les

threads clients doivent **être détachés** pour que le **thread principal** puisse **traiter les requêtes**

dès qu'elles lui parviennent. On ne connaît pas le nombre de threads actifs à un moment donné.

3. Modèle Producteur-Consommateur en pool



- **Utilisation d'un pool de threads :**

fait partie d'un

groupe ("pool") ou **groupe de processus** de threads préalablement **créés au démarrage** de

l'application serveur → sont en attente de se voir affecter le traitement d'une requête.

Ici pas de danger d'une inflation du nombre de threads.

REMARQUE : Qu'on utilise la 1^{ère} ou la 2^e alternative, le **thread principal** fait bien office de **producteur de tâches** tandis que **les threads** sont des **consommateurs de ces tâches**.

Chacun des **threads consommateurs** ignore ce que font les autres; ils dépendent tous du

serveur maître qui **les a créés** : ils en sont donc **les esclaves**.

les

différents threads exécutent tous obligatoirement le même code, d'où l'importance de disposer de code réentrant.

Éléments de synchronisation nécessaires

Le programme utilise plusieurs mécanismes de synchronisation pour assurer la coordination entre les threads :

1. Mutex (verrous) :

- `pthread_mutex_t mutexNbreActivites;`
- `pthread_mutex_t mutexCptNomsClients;`

Ces mutex protègent les variables globales `nbreNomsClientsNonTraites` et `cptNomsClients` pour éviter les conditions de course.

2. Variables de condition :

- `pthread_cond_t condNbreActivites;`
- `pthread_cond_t condCptNomsClients;`

Elles permettent aux threads d'attendre qu'une condition spécifique soit remplie avant de poursuivre l'exécution.

- `condCptNomsClients` est utilisée pour réveiller un thread client lorsqu'un nouveau nom est disponible.
- `condNbreActivites` est utilisée pour indiquer au thread principal que tous les noms ont été traités.

Code des fonctions de threads

Thread principal (`main`) :

- **Paradigme** : producteur-consommateur avec un pool de threads.
- **Boucle infinie ?** Non.
- **Fonctions POSIX utilisées** :
 - `pthread_create()` pour créer les threads.
 - `pthread_detach()` pour détacher les threads et éviter des fuites de ressources.
 - `pthread_mutex_init()` , `pthread_cond_init()` pour initialiser les mutex et les variables de condition.
 - `pthread_mutex_lock()` , `pthread_mutex_unlock()` pour la synchronisation.
 - `pthread_cond_wait()` , `pthread_cond_signal()` pour la coordination des threads.

Thread de traitement (`fctThread()`) :

- **Paradigme** : consommateur dans le modèle producteur-consommateur.
- **Boucle infinie ?** Oui (le thread boucle continuellement pour traiter les requêtes).
- **Fonctions POSIX utilisées** :

- `pthread_mutex_lock()` et `pthread_mutex_unlock()` pour protéger l'accès aux variables globales.
- `pthread_cond_wait()` pour attendre qu'un nom soit disponible.
- `pthread_cond_signal()` pour notifier l'avancement du traitement.

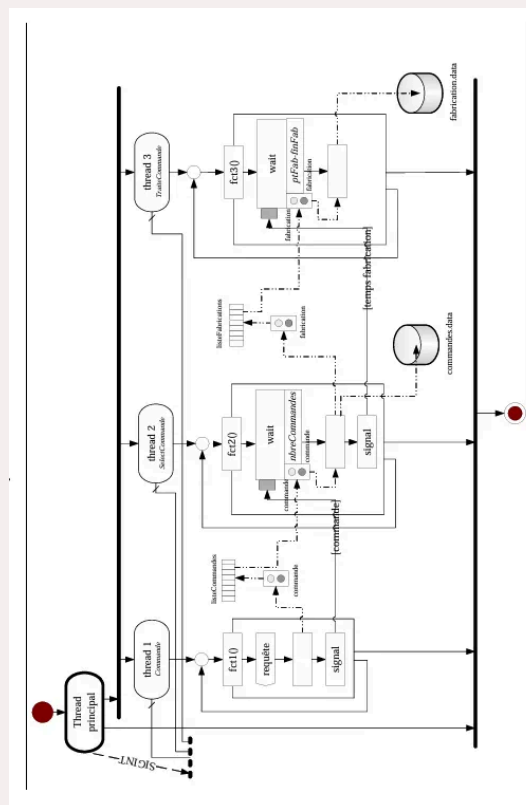
4

. Modèle Pipeline

Explication générale

Le modèle du pipeline consiste à diviser une tâche en plusieurs étapes traitées par des threads successifs, chaque thread travaillant sur des données à différents stades de traitement.

schéma



Éléments de synchronisation

- Mutex (pour protéger les accès concurrentiels aux structures de données partagées)
- Variables de condition (pour signaler aux threads qu'une nouvelle tâche est disponible)

Code des fonctions de threads

Résumé des éléments communs :

- **Paradigme** : Boucle infinie, chaque thread attend un événement, effectue son traitement, puis passe à l'étape suivante.
- **Librairies POSIX** : Utilisation de `pthread_mutex_lock`, `pthread_cond_signal`, et `pthread_cond_wait` pour la synchronisation.

Thread Commande

- **Rôle** : Gère l'entrée des commandes et les place dans une structure de données partagée, puis déclenche le thread suivant.

Thread SelectCommande

- **Rôle** : Récupère les commandes, les écrit dans un fichier et envoie la commande à l'étape suivante.

Thread TraiteCommande

- **Rôle** : Traite les commandes et génère le résultat final, tout en écrivant dans un fichier de log.